

Zerocoin Studios - Internship Selection - Exercise report

Technical Approach:

- My image processing pipeline code is separated into 3 script files.
 - **utils.py** containing all **reusable, generalised**, modular code that can be used in future pipelines.
 - **Gaussian_blur_and_tone_mapping.cl** has the 2 required kernels described in them.
 - **pipeline.py** has the pipeline script which is specific for this application and **not reusable**.
- The entire pipeline is designed in a **data coupled manner** (Tightly coupled). Almost all the function blocks in utils.py are self **contained** but every next stage requires a specific type of data arguments to be returned from the previous stage. It is generalised to an extent that it can take **any .cl kernel file with multiple kernels** and implement all its kernels **sequentially**. Obviously, there is one function **execute_kernels_dynamic()** that is dependent on other functions in the utils.py file.
- **Why?** : It has **less computational overhead** at each function level. As arguments are supplied as required. And I had **experience** working with this tightly data coupled approach before.

Kernel Design Approach:

- **Gaussian blur** : A standard 3×3 Gaussian blur kernel (normalized) is applied across the image. For each output pixel, the kernel computes a weighted average of its 3×3 neighborhood using the predefined weights.
- **Boundary Handling Strategy**: The kernel uses the OpenCL **CLK_ADDRESS_CLAMP** mode via the sampler. This means that when pixel coordinates fall outside the image boundary (e.g., near edges), they are clamped to the **nearest valid edge** coordinate. .
→ For example, if the kernel asks for a pixel at $(-1, y)$, it reads from $(0, y)$ instead.
- **Why?** : Clamping **avoids artifacts** like black borders or undefined values that could result from out-of-bound reads. It's efficient, hardware-supported, and well-suited for small kernels like our 3×3 kernel.

- **Logarithmic Tone Mapping and Color Ratio Preservation:**

$$Y_{\text{out}} = \frac{\log(1 + Y)}{\log(1 + Y_{\text{max}})}$$

- **Tone Mapping Formula:**
where Y is the original luminance and Ymax is a global **scene parameter** representing the maximum expected luminance. This **logarithmic compression** maps higher luminance values into a compressed range [0,1].
- **Color Ratio Preservation**: To maintain visual color fidelity, the output RGB is scaled proportionally:

$$\text{rgb}_{\text{out}} = \left(\frac{Y_{\text{out}}}{Y} \right) \cdot \text{rgb}$$

This ensures that the **chromaticity (color ratios)** of the original pixel is preserved, while only the **intensity (luminance)** is altered. If Y=0, RGB is set to zero to avoid division-by-zero.

Challenges, Learning, & Problem-Solving :

- **Challenges** : learning and implementing **C99-based kernel programming language**.
- **Learning** : Essentials of opencl and kernel programming. And inner workings of a basic image pipeline.
- **Systematic approach** : Spent time going through documentation and watching tutorials and more GPT 😊

Parallelism & Performance: (Other things are well commented in the code script files). 1 highlightable choice,

- **Work-Group Size**: Left it automatic (default) so it performs efficiently in all different compute platforms.

Project Closure & Future Work :

- **Strategy**: Initially made a basic working (**not so neat, trial & error**) pipeline in a .ipynb file. It served as a reference and backbone in modularizing and arriving at final source code.
- **Future** : The current blur kernel performs **9 global memory reads per pixel**. Maybe explore other options like using **OpenCL local memory** to load a **shared tile** into fast on-chip memory. reducing memory traffic.